

Algoritmos de ordenamiento

Algoritmos y Estructuras de Datos II -
Estructuras de Datos y Algoritmos
Dr. Pablo Ponzio
Universidad Nacional de Río Cuarto
CONICET



Sorting

Sorting es la tarea de organizar una colección de datos según un orden dado

- Podemos realizar sorting sobre cualquier tipo de datos con un orden: **int**, **char**, **Integer**, **Strings**, etc
- En general, el algoritmo de sorting acomoda los elementos de forma ascendente o descendente
- Es importante que los algoritmos de sorting sean **eficientes** ya que en la práctica se quiere ordenar una cantidad grande de elementos

Sorting en JAVA

En Java se utiliza la interface **Comparable**:

```
public interface Comparable<T> {  
    int compareTo(T o);  
}
```

- Toda clase con un orden hereda de comparable.
- Permite implementar algoritmos de sorting polimórficos.
- La clase comparable provee un método CompareTo():

$\text{this.compareTo}(x) > 0 \equiv \text{this} > x$

$\text{this.compareTo}(x) < 0 \equiv \text{this} < x$

$\text{this.compareTo}(x) = 0 \equiv \text{this} = x$

usualmente compareTo(x)
devuelve -1, 0, 1

Comparable: Ejemplo

```
public class Employee implements Comparable<Employee> {  
    String firstName;  
    String lastName;  
  
    /**  
     * @post Compares this object with the specified object for order.  
     * Returns a negative integer, zero, or a positive integer as this  
     * object is less than, equal to, or greater than the specified object.  
     */  
    public int compareTo(Employee other)  
    {  
        if (this.lastName.compareTo(other.lastName) != 0) {  
            // If lastNames are different, compare lastName  
            return this.lastName.compareTo(other.lastName);  
        } else {  
            // If lastNames are the same, compare firstName  
            return this.firstName.compareTo(other.firstName);  
        }  
    }  
}
```

Importante...

Para analizar un algoritmo de sorting podemos tener en cuenta:

- **Eficiencia:** el tiempo de ejecución del algoritmo, en el peor caso, y también en el caso promedio.
- **Cantidad de comparaciones:** Cuantas veces comparamos para ordenar los elementos.
- **Cantidad de Intercambios:** Cuantos intercambios se realizan para ordenar

Las comparaciones pueden ser costosas

Los intercambios son constantes en JAVA

Estabilidad

Un algoritmo de sorting se dice estable si preserva el orden de los elementos con las mismas claves

- La clave es el campo o el atributo sobre el cual ordenamos.
- Hay algoritmos que son estables y otros no.
- En general cualquier algoritmo se puede hacer estable con algún costo extra: agregar más claves, etc.

Class base para Sorting

```
/**
 * Base class for sorting algorithms.
 */
public abstract class Sorting
{
    /**
     * @post Rearranges the array a in ascending order.
     */
    public abstract void sort(Comparable[] a);

    /**
     * @post Returns true iff v is smaller than w.
     */
    protected boolean less(Comparable v, Comparable w) {
        return v.compareTo(w) < 0;
    }

    /**
     * @pre 0<=i<a.length && 0<=j<a.length
     * @post Swaps the elements in positions i and j of a.
     */
    protected static void exch(Object[] a, int i, int j) {
        Object swap = a[i];
        a[i] = a[j];
        a[j] = swap;
    }
}
```


Selection sort

- Elegir el elemento más chico del arreglo e intercambiarlo con el elemento en la primera posición
- Elegir el segundo elemento más chico del arreglo e intercambiarlo con el elemento en la segunda posición
- y así sucesivamente...

		a[]										
i	min	0	1	2	3	4	5	6	7	8	9	10
		S	O	R	T	E	X	A	M	P	L	E
0	6	S	O	R	T	E	X	A	M	P	L	E
1	4	A	O	R	T	E	X	S	M	P	L	E
2	10	A	E	R	T	O	X	S	M	P	L	E
3	9	A	E	E	T	O	X	S	M	P	L	R
4	7	A	E	E	L	O	X	S	M	P	T	R

...

Selection sort

- Invariante: Selection sort mantiene ordenado el arreglo $a[0 \dots i-1]$
- En cada iteración, intercambia el elemento en i por el elemento más chico en $a[i \dots n-1]$
 - De esta manera, se obtiene que el arreglo $a[0 \dots i]$ está ordenado

Iteración i:

i	min	0	1	2	3	4	5	6	7	8	9	10
4	7	A	E	E	L	O	X	S	M	P	T	R

a[0...i-1] ordenado

Iteración $i+1$:

5 7 A E E L M X S **O** P T R

a[0...i] ordenado

- Encapsular los algoritmos en objetos nos va a permitir intercambiar las implementaciones de los algoritmos de Sorting sin modificar los clientes
 - Esto se corresponde con el patrón de diseño llamado Strategy
- Para ordenar un arreglo con selection sort crearemos un objeto de la clase Selection:
 - `Sorting s = new Selection();`
`s.sort(arr);`
- Para ordenar con insertion sort ejecutaremos:
 - `Sorting s = new Insertion();`
`s.sort(arr);`

```
public class Selection extends Sorting {

    /**
     * @post Creates an object that encapsulates the selection
     *       sort algorithm.
     */
    public Selection() { }

    /**
     * @post Rearranges the array a in ascending order
     *       using the selection sort algorithm.
     */
    public void sort(Comparable[] a) {
        int n = a.length;
        for (int i = 0; i < n; i++) {
            // Invariant: a[0...i-1] is sorted
            assert isSorted(a, 0, i-1);
            // Exchange a[i] with smallest entry in a[i...n-1]
            int min = i;
            for (int j = i+1; j < n; j++) {
                if (less(a[j], a[min]))
                    min = j;
            }
            exch(a, i, min);
            // Now a[0...i] is sorted
            assert isSorted(a, 0, i);
        }
        assert isSorted(a);
    }
}
```

- Encapsular los algoritmos en objetos nos va a permitir intercambiar las implementaciones de los algoritmos de Sorting sin modificar los clientes
 - Esto se corresponde con el patrón de diseño llamado Strategy
- Para ordenar un arreglo con selection sort crearemos un objeto de la clase Selection:
 - Sorting s = new Selection();

```
public class Selection extends Sorting {

    /**
     * @post Creates an object that encapsulates the selection
     *       sort algorithm.
     */
    public Selection() { }

    /**
     * @post Rearranges the array a in ascending order
     *       using the selection sort algorithm.
     */
    public void sort(Comparable[] a) {
        int n = a.length;
        for (int i = 0; i < n; i++) {
            // Invariant: a[0...i-1] is sorted
            assert isSorted(a, 0, i-1);
            // Exchange a[i] with smallest entry in a[i...n-1]
            int min = i;
            for (int j = i+1; j < n; j++) {
                if (less(a[j], a[min]))
                    min = j;
            }
            exch(a, i, min);
            // a[0...i] is sorted
            assert isSorted(a, 0, i);
        }
        assert isSorted(a);
    }
}
```

```
@Test
public void test0()
{
    Integer [] arr = new Integer [] {3,2,1};
    Integer [] expected = new Integer [] {1,2,3};
    Sorting s = new Selection();
    s.sort(arr);
    assertTrue(Arrays.equals(arr, expected));
    assertTrue(s.isSorted(arr));
}
```

- Encapsular los algoritmos en objetos nos va a permitir intercambiar las implementaciones de los algoritmos de Sorting sin modificar los clientes
 - Esto se corresponde con el patrón de diseño llamado Strategy
- Para ordenar un arreglo con selection sort crearemos un objeto de la clase Selection:
 - Sorting s = new Selection();

```
public class Selection extends Sorting {

    /**
     * @post Creates an object that encapsulates the selection
     *       sort algorithm.
     */
    public Selection() { }

    /**
     * @post Rearranges the array a in ascending order
     *       using the selection sort algorithm.
     */
    public void sort(Comparable[] a) {
        int n = a.length;
        for (int i = 0; i < n; i++) {
            // Invariant: a[0...i-1] is sorted
            assert isSorted(a, 0, i-1);
            // Exchange a[i] with smallest entry in a[i...n-1]
            int min = i;
            for (int j = i+1; j < n; j++) {
                if (a[j].compareTo(a[min]) < 0)
                    min = j;
            }
            swap(a, i, min);
        }
    }
}
```

```
@Test
public void test0()
{
    Integer [] arr = new Integer [10];
    Integer [] expected = new Integer [10];
    Sorting s = new Selection();
    s.sort(arr);
    assertTrue(Arrays.equals(arr, expected));
    assertTrue(s.isSorted(arr));
}
```

```
@Test
public void test1()
{
    Integer [] arr = new Integer [10] {2,3,1,5,6,9,6,3};
    Integer [] expected = new Integer [10] {1,2,3,3,5,6,6,9};
    Sorting s = new Selection();
    s.sort(arr);
    assertTrue(Arrays.equals(arr, expected));
    assertTrue(s.isSorted(arr));
}
```

Selection sort: Características

- El peor caso es cuando el arreglo está ordenado en forma decreciente
- El tiempo de ejecución en el peor caso es:

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} c \in \theta(n^2)$$

- La cantidad de comparaciones que realiza en el peor caso es:

$$\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} 1 \in \theta(n^2)$$

- La cantidad de intercambios que realiza en el peor caso es:

$$T(n) = \sum_{i=0}^{n-1} 1 \in \theta(n)$$

- Esta implementación es estable

```
public void sort(Comparable[] a) {  
    int n = a.length;  
    for (int i = 0; i < n; i++) {  
        // Invariant: a[0...i-1] is sorted  
        assert isSorted(a, 0, i-1);  
        // Exchange a[i] with smallest entry in a[i...n-1]  
        int min = i;  
        for (int j = i+1; j < n; j++) {  
            if (less(a[j], a[min]))  
                min = j;  
        }  
        exch(a, i, min);  
        // Now a[0...i] is sorted  
        assert isSorted(a, 0, i);  
    }  
    assert isSorted(a);  
}
```

Bubble sort

- Intercambiar los pares de elementos adyacentes en $a[n-1 \dots 0]$ que no estén ordenados
 - Esto deja el elemento más chico en $a[0]$
- Intercambiar los pares de elementos adyacentes en $a[n-1 \dots 1]$ que no estén ordenados
 - Esto deja el segundo elemento más chico en $a[1]$
- y así sucesivamente...

```
[S, O, R, T, E, X, A, M, P, L, E]  
[A, S, O, R, T, E, X, E, M, P, L]  
[A, E, S, O, R, T, E, X, L, M, P]  
[A, E, E, S, O, R, T, L, X, M, P]  
[A, E, E, L, S, O, R, T, M, X, P]
```

Bubble sort

- Invariante: Bubble sort mantiene ordenado el arreglo $a[0...i-1]$
- En cada iteración, intercambia los pares de elementos adyacentes en $a[n-1...i]$ que no están ordenados, y así logra que el menor de $a[n-1...i]$ quede en $a[i]$
 - De esta manera, se obtiene que el arreglo $a[0...i]$ está ordenado

Iteración i :

[A, E, E, L, S, O, R, T, M, X, P]
 $a[0...i-1]$ ordenado

Iteración $i+1$:

[A, E, E, L, M, S, O, R, T, P, X]
 $a[0...i]$ ordenado


```

public class Bubble extends Sorting {

    /**
     * @post Creates an object that encapsulates the bubble
     *       sort algorithm.
     */
    public Bubble() { }

    /**
     * @post Rearranges the array a in ascending order
     *       using the bubble sort algorithm.
     */
    public void sort(Comparable[] a) {
        int n = a.length;
        for (int i = 0; i < n; i++) {
            // Invariant: a[0...i-1] is sorted
            int exchanges = 0;
            // Swap unsorted adjacent elements a[i...n-1]
            // This puts the smallest element from a[i...n-1] in a[i]
            for (int j = n-1; j > i; j--) {
                if (less(a[j], a[j-1])) {
                    exch(a, j, j-1);
                    exchanges++;
                }
            }
            // a[0...i] is now sorted
            assert isSorted(a, 0, i);
            if (exchanges == 0) break;
        }
        assert isSorted(a);
    }
}

```


Bubble sort: Tests

- Como están programados a partir de la misma clase abstracta Sorting, los tests son muy similares para todos los algoritmos de Sorting
- Esto indica que luego podremos cambiar fácilmente la implementación de los algoritmos de Sorting en los clientes (si están programados respecto de la clase abstracta Sorting)

```
@Test
public void test0()
{
    Integer [] arr = new Integer [] {3,2,1};
    Integer [] expected = new Integer [] {1,2,3};
    Sorting s = new Selection();
    s.sort(arr);
    assertTrue(Arrays.equals(arr, expected));
    assertTrue(s.isSorted(arr));
}
```

```
@Test
public void test4()
{
    Integer [] arr = new Integer [] {3,2,1};
    Integer [] expected = new Integer [] {1,2,3};
    Sorting s = new Bubble();
    s.sort(arr);
    assertTrue(Arrays.equals(arr, expected));
    assertTrue(s.isSorted(arr));
}
```

Bubble sort: Características

- El peor caso es cuando el arreglo está ordenado en forma decreciente
- El tiempo de ejecución en el peor caso es:

$$• T(n) = \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} c \in \theta(n^2)$$

- La cantidad de comparaciones que realiza en el peor caso es:

$$• \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} 1 \in \theta(n^2)$$

- La cantidad de intercambios que realiza en el peor caso es:

$$• \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} 1 \in \theta(n^2)$$

- Esta implementación es estable

```
public void sort(Comparable[] a) {  
    int n = a.length;  
    for (int i = 0; i < n; i++) {  
        // Invariant: a[0...i-1] is sorted  
        int exchanges = 0;  
        // Swap unsorted adjacent elements a[i...n-1]  
        // This puts the smallest element from a[i...n-1] in a[i]  
        for (int j = n-1; j > i; j--) {  
            if (less(a[j], a[j-1])) {  
                exch(a, j, j-1);  
                exchanges++;  
            }  
        }  
        // a[0...i] is now sorted  
        assert isSorted(a, 0, i);  
        if (exchanges == 0) break;  
    }  
    assert isSorted(a);  
}
```

Insertion sort

- Insertar el elemento en la posición 1 ordenado en $a[0...1]$
- Insertar el elemento en la posición 2 ordenado en $a[0...2]$
- Insertar el elemento en la posición 3 ordenado en $a[0...3]$
- y así sucesivamente

[S, O, R, T, E, X, A, M, P, L, E]

[O, S, R, T, E, X, A, M, P, L, E]

[O, R, S, T, E, X, A, M, P, L, E]

[O, R, S, T, E, X, A, M, P, L, E]

...

Insertion sort

- Invariante: Insertion sort mantiene ordenado el arreglo $a[0\dots i-1]$
- En cada iteración, inserta el elemento en i de manera ordenada en $a[0\dots i]$
 - De esta manera, se obtiene que el arreglo $a[0\dots i]$ está ordenado

Iteración i :

[O, R, S, T, E, X, A, M, P, L, E]
 $a[0\dots i-1]$ ordenado

Iteración $i+1$:

[E, O, R, S, T, X, A, M, P, L, E]
 $a[0\dots i]$ ordenado

- Encapsular los algoritmos en objetos nos va a permitir intercambiar las implementaciones de los algoritmos de Sorting sin modificar los clientes
 - Esto se corresponde con el patrón de diseño llamado Strategy
- Para ordenar un arreglo con selection sort crearemos un objeto de la clase Selection:
 - `Sorting s = new Selection();`
`s.sort(arr);`
- Para ordenar con insertion sort ejecutaremos:
 - `Sorting s = new Insertion();`
`s.sort(arr);`

```
public class Insertion extends Sorting {

    /**
     * @post Creates an object that encapsulates the
     *        insertion sort algorithm.
     */
    public Insertion() { }

    /**
     * @post Rearranges the array a in ascending order
     *        using the insertion sort algorithm.
     */
    public void sort(Comparable[] a) {
        int n = a.length;
        for (int i = 1; i < n; i++) {
            // Invariant: a[0...i-1] is sorted
            assert isSorted(a, 0, i-1);
            // Insert a[i] in sorted order in a[0...i]
            for (int j = i; j > 0 && less(a[j], a[j-1]); j--)
            {
                exch(a, j, j-1);
            }
            // a[0...i] is now sorted
            assert isSorted(a, 0, i);
        }
        assert isSorted(a);
    }
}
```

- Encapsular los algoritmos en objetos nos va a permitir intercambiar las implementaciones de los algoritmos de Sorting sin modificar los clientes
 - Esto se corresponde con el patrón de diseño llamado Strategy
- Para ordenar un arreglo con selection sort crearemos un objeto de la clase Selection:
 - `Sorting s = new Selection();`
`s.sort(arr);`

```
public class Insertion extends Sorting {
```

```
    /**
     * @post Creates an object that encapsulates the
     *        insertion sort algorithm.
     */
    public Insertion() { }

    /**
     * @post Rearranges the array a in ascending order
     *        using the insertion sort algorithm.
     */
    public void sort(Comparable[] a) {
        int n = a.length;
        for (int i = 1; i < n; i++) {
            // Invariant: a[0...i-1] is sorted
            assert isSorted(a, 0, i-1);
            // Insert a[i] in sorted order in a[0...i]
            for (int j = i; j > 0 && less(a[j], a[j-1]); j--)
            {
                exch(a, j, j-1);
            }
        }
    }
}
```

```
@Test
public void test0()
{
    Integer [] arr = new Integer [] {3,2,1};
    Integer [] expected = new Integer [] {1,2,3};
    Sorting s = new Selection();
    s.sort(arr);
    assertTrue(Arrays.equals(arr, expected));
    assertTrue(s.isSorted(arr));
}
```

```
    // Invariant: a[0...i] is now sorted
    assert isSorted(a, 0, i);
}
```

- Encapsular los algoritmos en objetos nos va a permitir intercambiar las implementaciones de los algoritmos de Sorting sin modificar los clientes
 - Esto se corresponde con el patrón de diseño llamado Strategy
- Para ordenar un arreglo con selection sort crearemos un objeto de la clase Selection:
 - `Sorting s = new Selection();`
`s.sort(arr);`

```
public class Insertion extends Sorting {
```

```
    /**
     * @post Creates an object that encapsulates the
     *        insertion sort algorithm.
     */
    public Insertion() { }

    /**
     * @post Rearranges the array a in ascending order
     *        using the insertion sort algorithm.
     */
    public void sort(Comparable[] a) {
        int n = a.length;
        for (int i = 1; i < n; i++) {
            // Invariant: a[0...i-1] is sorted
            assert isSorted(a, 0, i-1);
            // Insert a[i] in sorted order in a[0...i]
            for (int j = i; j > 0 && less(a[j], a[j-1]); j--)
            {
                exch(a, i, i-1);
            }
        }
    }
}
```

```
@Test
public void test0()
{
    Integer [] arr = new Integer [] {3,2,1};
    Integer [] expected = new Integer [] {1,2,3};
    Sorting s = new Selection();
    s.sort(arr);
    assertTrue(Arrays.equals(arr, expected));
    assertTrue(s.isSorted(arr));
}
```

```
@Test
public void test4()
{
    Integer [] arr = new Integer [] {3,2,1};
    Integer [] expected = new Integer [] {1,2,3};
    Sorting s = new Bubble();
    s.sort(arr);
    assertTrue(Arrays.equals(arr, expected));
    assertTrue(s.isSorted(arr));
}
```


Insertion sort: Características

- El peor caso es cuando el arreglo está ordenado en forma decreciente
- El tiempo de ejecución en el peor caso es:

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=1}^i c \in \theta(n^2)$$

- La cantidad de comparaciones que realiza en el peor caso es:

$$\sum_{i=0}^{n-1} \sum_{j=1}^i 1 \in \theta(n^2)$$

- La cantidad de intercambios que realiza en el peor caso es:

$$\sum_{i=0}^{n-1} \sum_{j=1}^i 1 \in \theta(n^2)$$

```
public void sort(Comparable[] a) {  
    int n = a.length;  
    for (int i = 1; i < n; i++) {  
        // Invariant: a[0...i-1] is sorted  
        assert isSorted(a, 0, i-1);  
        // Insert a[i] in sorted order in a[0...i]  
        for (int j = i; j > 0 && less(a[j], a[j-1]); j--) {  
            exch(a, j, j-1);  
        }  
        // a[0...i] is now sorted  
        assert isSorted(a, 0, i);  
    }  
    assert isSorted(a);  
}
```


Insertion sort: Características

- Esta implementación es estable
- En el mejor caso es:
 - $T(n) = \sum_{i=0}^{n-1} c \in \theta(n)$
- En el caso promedio es cuadrático, y se lo considera ineficiente para ser usado en la práctica

```
public void sort(Comparable[] a) {  
    int n = a.length;  
    for (int i = 1; i < n; i++) {  
        // Invariant: a[0...i-1] is sorted  
        assert isSorted(a, 0, i-1);  
        // Insert a[i] in sorted order in a[0...i]  
        for (int j = i; j > 0 && less(a[j], a[j-1]); j--) {  
            exch(a, j, j-1);  
        }  
        // a[0...i] is now sorted  
        assert isSorted(a, 0, i);  
    }  
    assert isSorted(a);  
}
```

Mergesort

- Mergesort es un algoritmo divide and conquer
- Idea:
 - Dividir el arreglo en dos mitades, y ordenar recursivamente cada una de las mitades
 - Merge: Dadas las dos mitades ordenadas, poner los elementos en el arreglo original de manera ordenada
 - Para hacerlo eficientemente debe ser $O(n)$, con n la cantidad de elementos en el arreglo
 - Iterativamente ir tomando el menor de los primeros elementos de cada arreglo (los menores de cada arreglo) y copiándolo al arreglo original

Mergesort: Ejemplo

[38, 27, 43, 3, 9, 82, 10]

Mergesort: Ejemplo

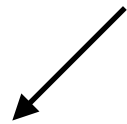
[38, 27, 43, 3, 9, 82, 10]



sort left [38, 27, 43]

Mergesort: Ejemplo

[38, 27, 43, 3, 9, 82, 10]



sort left [38, 27, 43]

La llamada recursiva ordena la parte izquierda!

Mergesort: Ejemplo

[38, 27, 43, 3, 9, 82, 10]



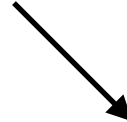
[27, 38, 43]

Mergesort: Ejemplo

[38, 27, 43, 3, 9, 82, 10]



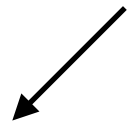
[27, 38, 43]



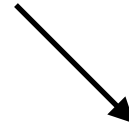
sort right [3, 9, 82, 10]

Mergesort: Ejemplo

[38, 27, 43, 3, 9, 82, 10]



[27, 38, 43]



sort right [3, 9, 82, 10]

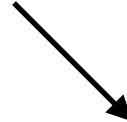
La llamada recursiva ordena la parte derecha!

Mergesort: Ejemplo

[38, 27, 43, 3, 9, 82, 10]

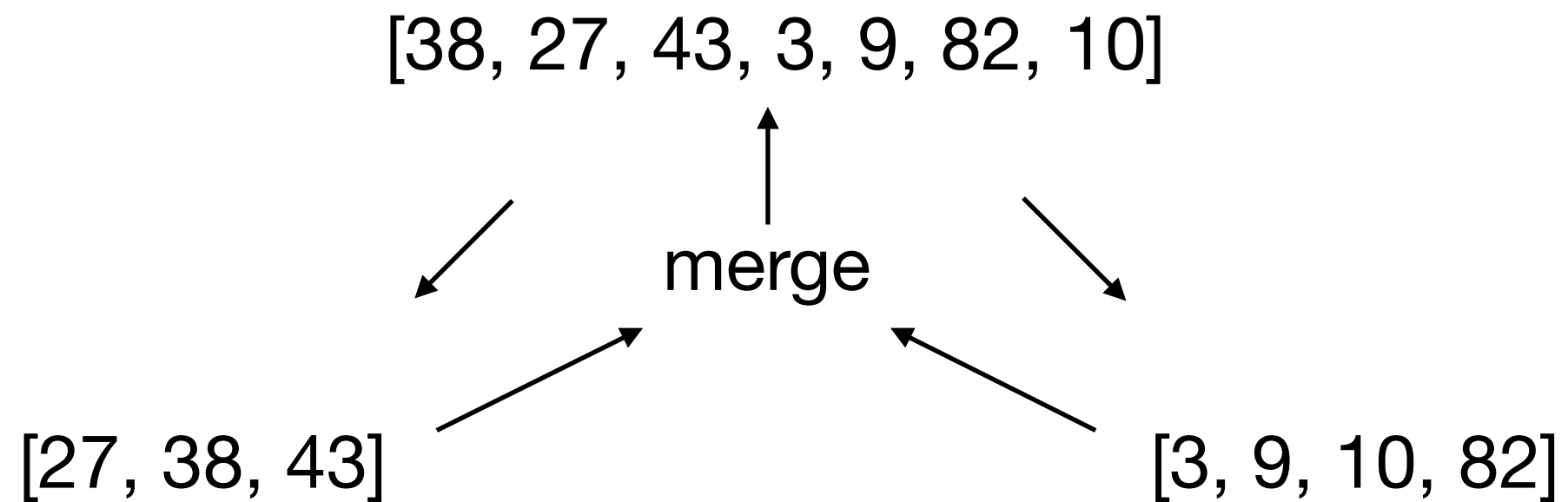


[27, 38, 43]

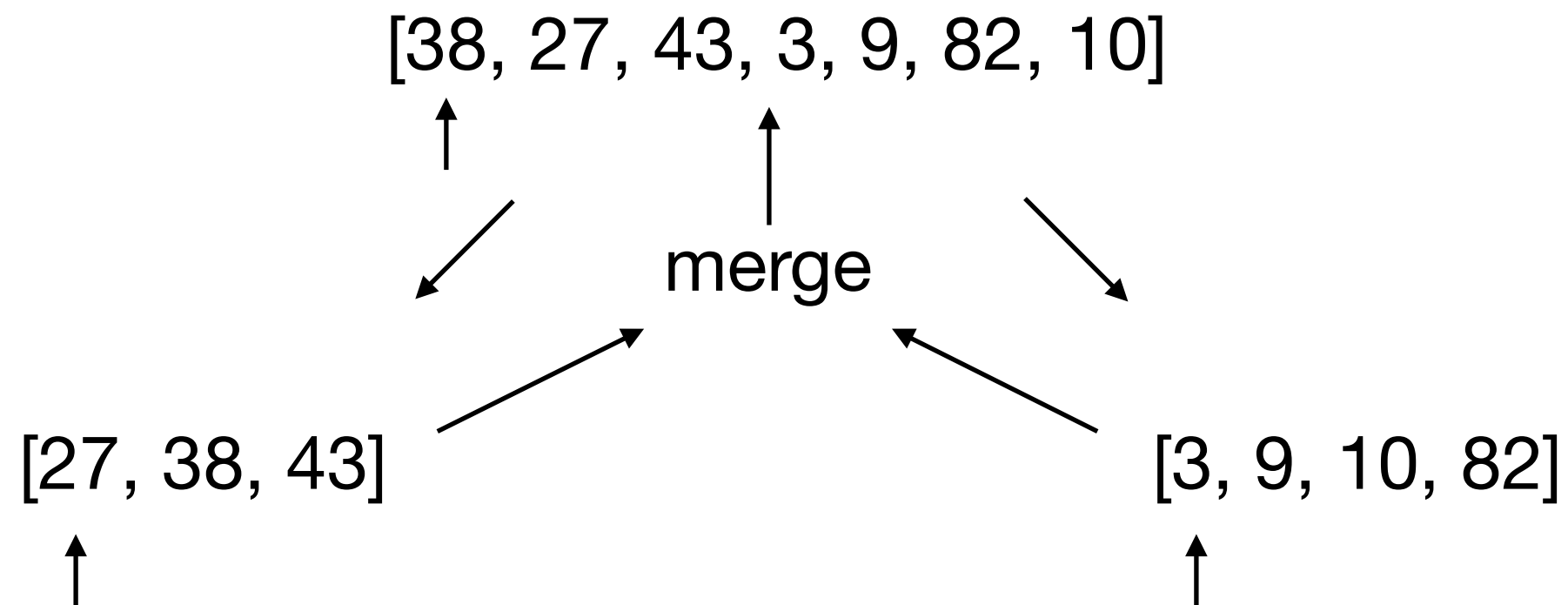


[3, 9, 10, 82]

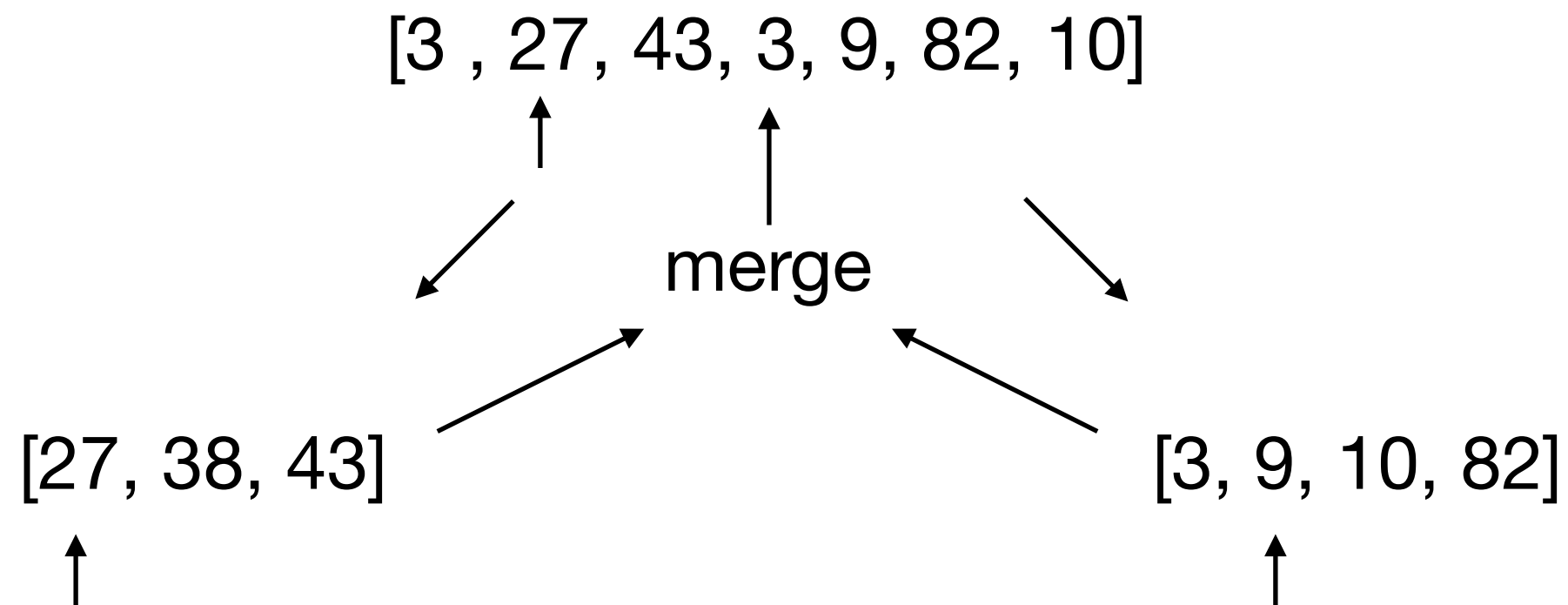
Mergesort: Ejemplo



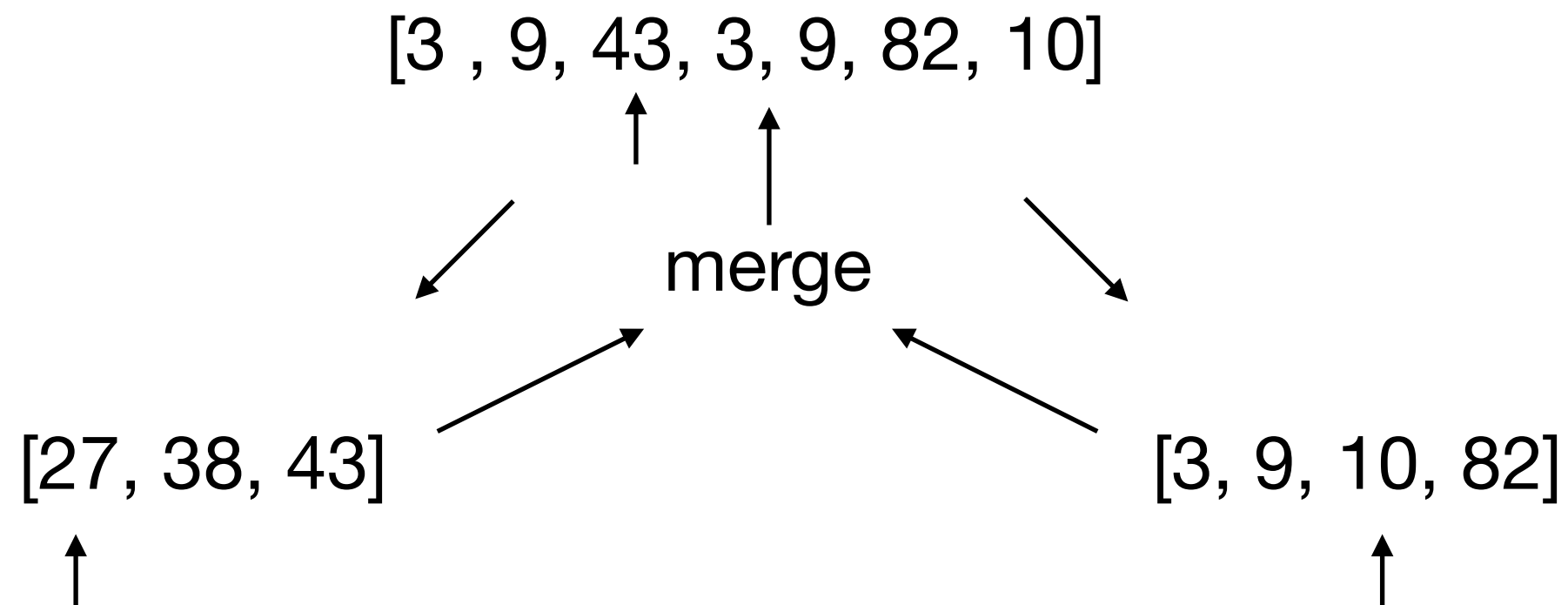
Mergesort: Ejemplo



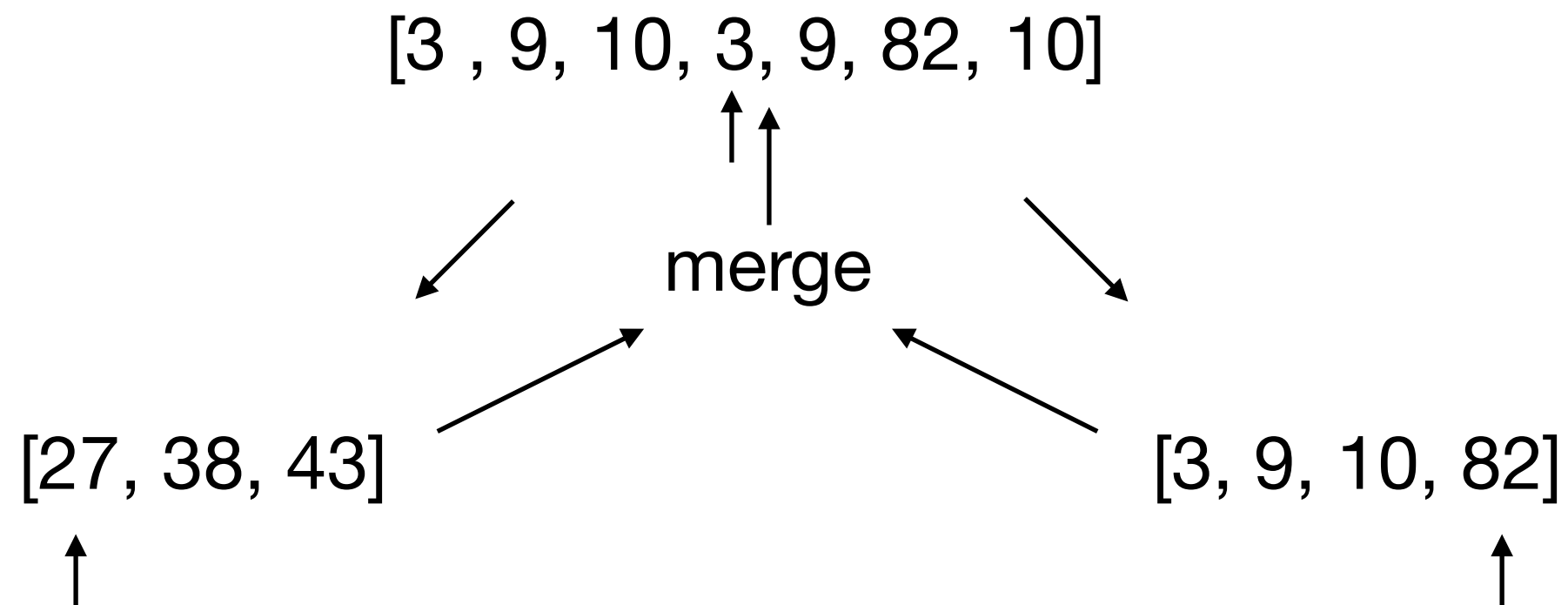
Mergesort: Ejemplo



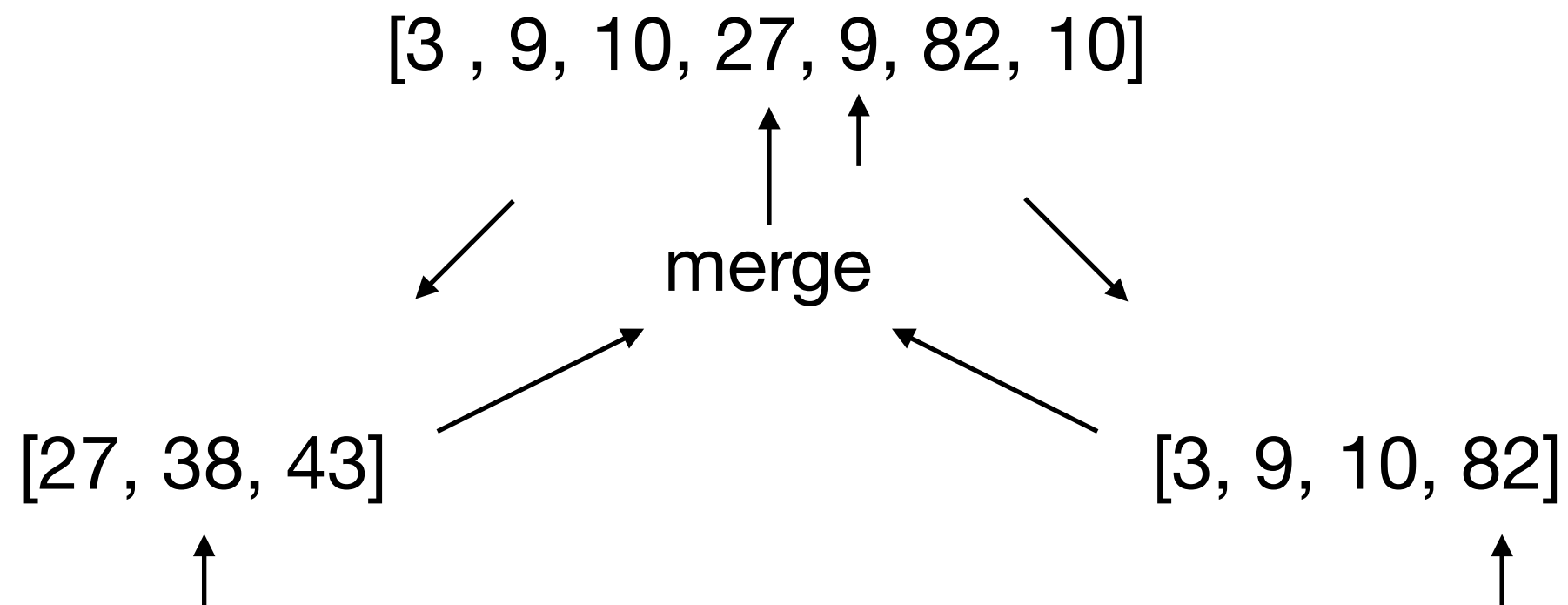
Mergesort: Ejemplo



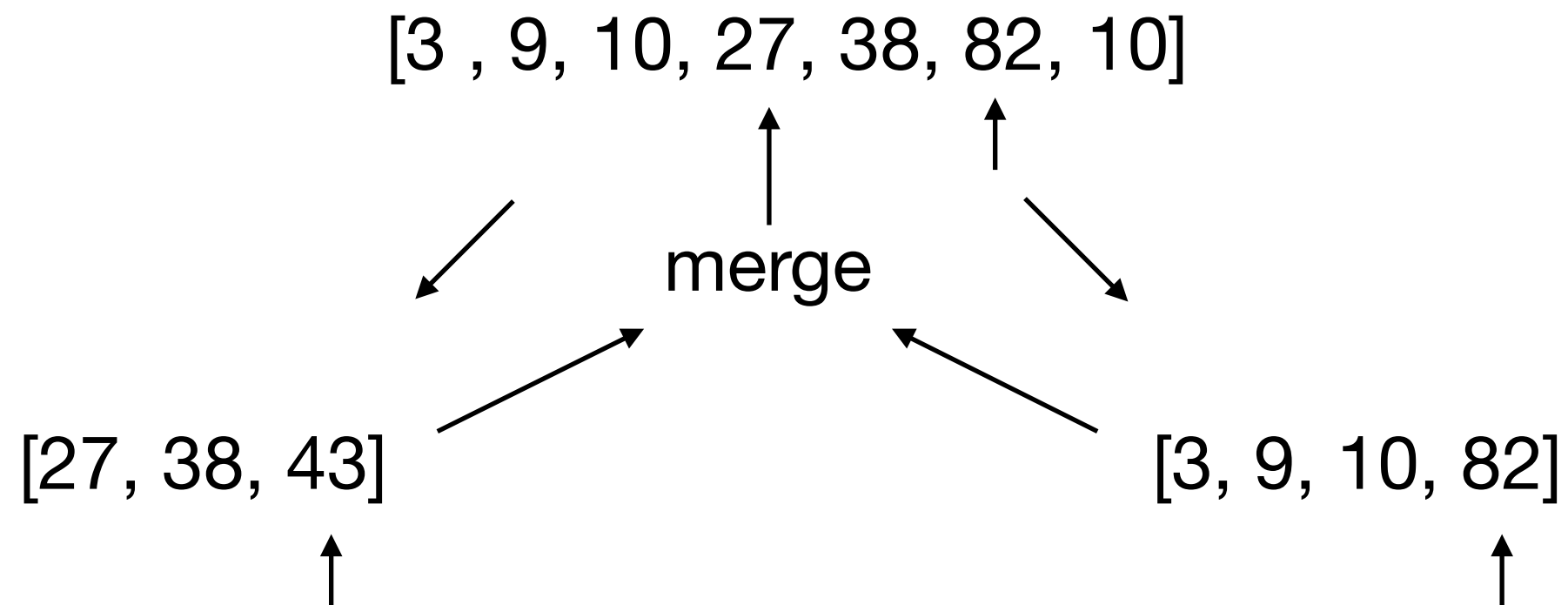
Mergesort: Ejemplo



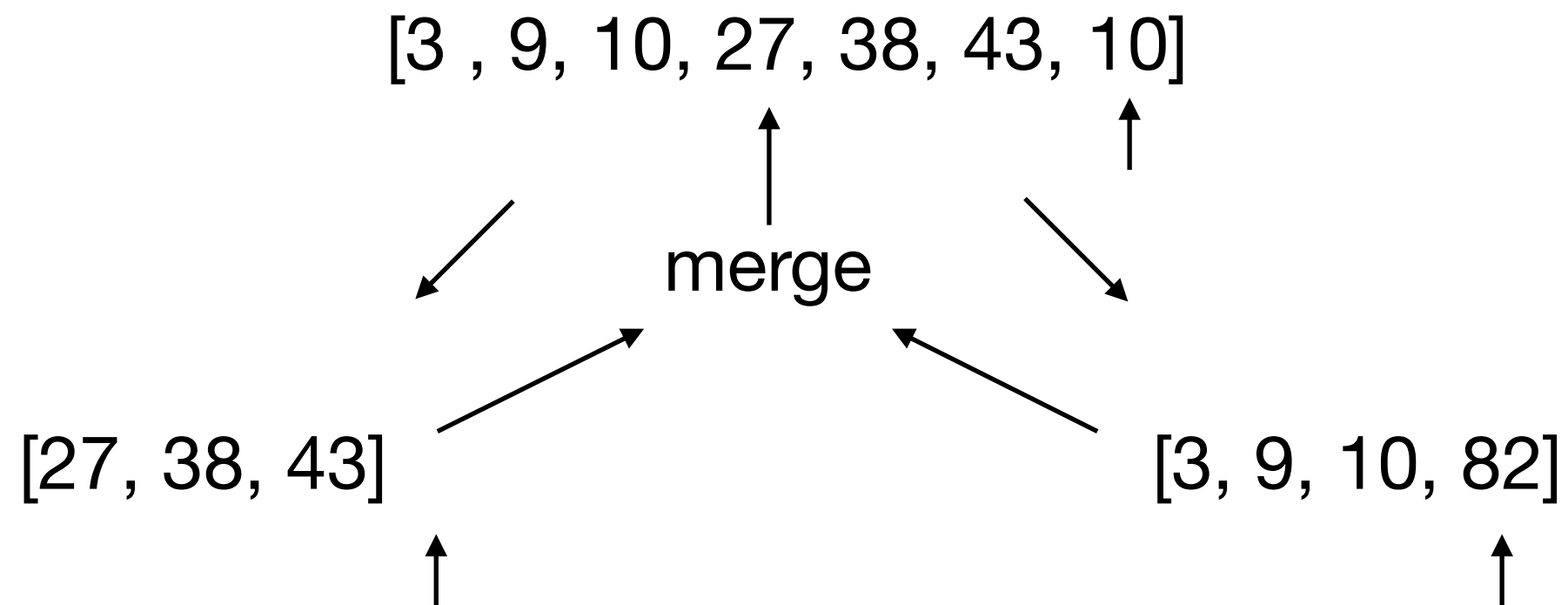
Mergesort: Ejemplo



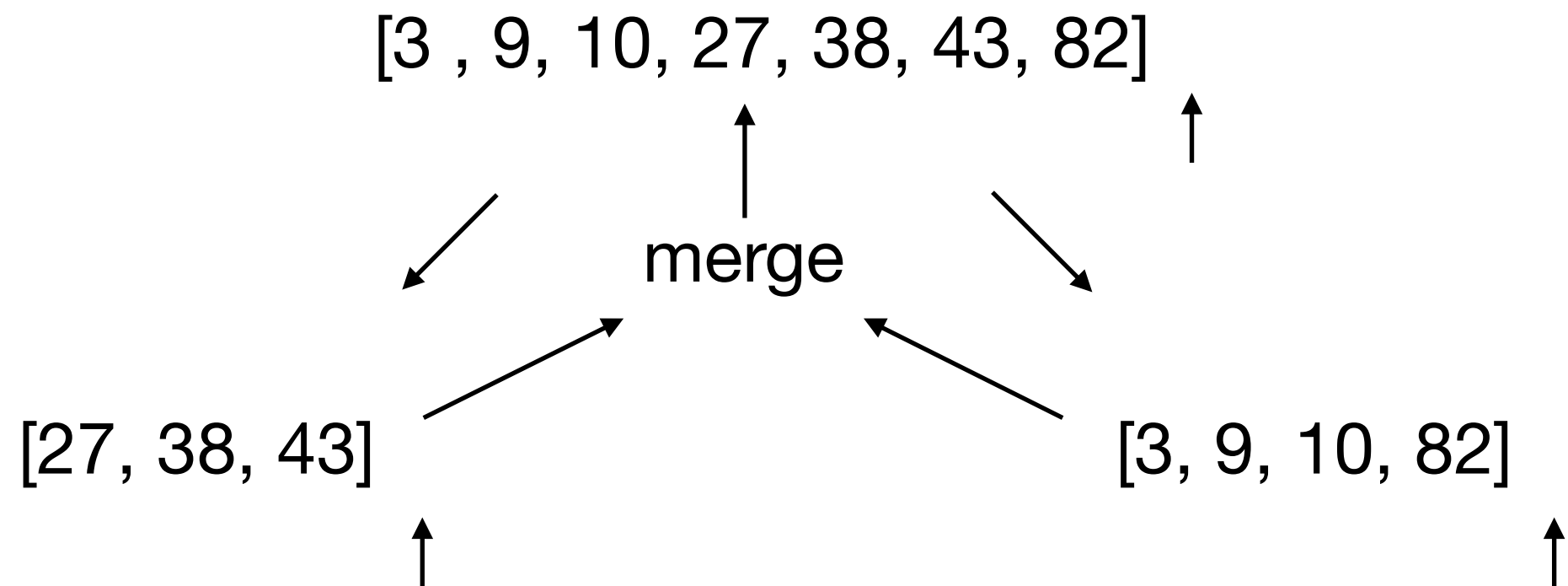
Mergesort: Ejemplo



Mergesort: Ejemplo



Mergesort: Ejemplo



Mergesort: Ejemplo

[3 , 9, 10, 27, 38, 43, 82]

Arreglo ordenado!

Mergesort

```
/**
 * @post Rearranges the array a in ascending order
 * using the mergesort algorithm.
 */
public void sort(Comparable[] arr) {
    // Base case: arrays of length 0 or 1 are already sorted
    if (arr.length <= 1) {
        return;
    }
    // 1. Divide into two halves
    int mid = arr.length / 2;
    Comparable[] left = copyRange(arr, 0, mid);
    Comparable[] right = copyRange(arr, mid, arr.length);
    // 2. Recursively sort both halves
    sort(left);
    sort(right);
    // 3. Merge back the two sorted halves to arr
    merge(left, right, arr);
    assert isSorted(arr);
}
```

Mergesort

```
/**
 * @post Rearranges the array a in ascending order
 * using the mergesort algorithm.
 */
public void sort(Comparable[] arr) {
    // Base case: arrays of length 0 or 1 are already sorted
    if (arr.length <= 1) {
        return;
    }
    // 1. Divide into two
    int mid = arr.length / 2;
    Comparable[] left = new Comparable[mid];
    Comparable[] right = new Comparable[arr.length - mid];
    // 2. Recursively sort
    sort(left);
    sort(right);
    // 3. Merge back the two sorted arrays
    merge(left, right, arr);
    assert isSorted(arr);
}
```

```
/**
 * @post Returns a copy of arr[from..to-1] in a
 * new array, where the first element is in
 * position 0 and the last element is in position
 * (to-from)-1.
 */
private Comparable[] copyRange(Comparable[] arr,
                                int from, int to) {
    Comparable[] copy = new Comparable[to - from];
    for (int i = 0; i < copy.length; i++) {
        copy[i] = arr[from + i];
    }
    return copy;
}
```

```

/**
 * @post Merges two sorted arrays left and right into one sorted array res
 */
private void merge(Comparable[] left, Comparable[] right, Comparable[] res) {
    // check precondition
    assert isSorted(left);
    assert isSorted(right);
    int i = 0, j = 0, k = 0;
    // Compare and take the smaller element
    while (i < left.length && j < right.length) {
        if (left[i].compareTo(right[j]) <= 0) {
            res[k] = left[i];
            i++;
        } else {
            res[k] = right[j];
            j++;
        }
        k++;
    }
    // Add any leftovers from the left array
    while (i < left.length) {
        res[k] = left[i];
        i++;
        k++;
    }
    // Add any leftovers from the right array
    while (j < right.length) {
        res[k] = right[j];
        j++;
        k++;
    }
    assert isSorted(res);
}

```

```

/**
 * @post Merges two sorted arrays left and right into one sorted array res
 */
private void merge(Comparable[] left, Comparable[] right, Comparable[] res) {
    // check precondition
    assert isSorted(left);
    assert isSorted(right);
    int i = 0, j = 0, k = 0;
    // Compare and take the smaller element
    while (i < left.length && j < right.length) {
        if (left[i].compareTo(right[j]) <= 0) {
            res[k] = left[i];
            i++;
        } else {
            res[k] = right[j];
            j++;
        }
        k++;
    }
    // Add any leftovers from the left array
    while (i < left.length) {
        res[k] = left[i];
        i++;
        k++;
    }
    // Add any leftovers from the right array
    while (j < right.length) {
        res[k] = right[j];
        j++;
        k++;
    }
    assert isSorted(res);
}

```

Poner en `res` el menor de los primeros elementos de cada arreglo (`i` y `j`)


```

/**
 * @post Merges two sorted arrays left and right into one sorted array res
 */
private void merge(Comparable[] left, Comparable[] right, Comparable[] res) {
    // check precondition
    assert isSorted(left);
    assert isSorted(right);
    int i = 0, j = 0, k = 0;
    // Compare and take the smaller element
    while (i < left.length && j < right.length) {
        if (left[i].compareTo(right[j]) <= 0) {
            res[k] = left[i];
            i++;
        } else {
            res[k] = right[j];
            j++;
        }
        k++;
    }
    // Add any leftovers from the left array
    while (i < left.length) {
        res[k] = left[i];
        i++;
        k++;
    }
    // Add any leftovers from the right array
    while (j < right.length) {
        res[k] = right[j];
        j++;
        k++;
    }
    assert isSorted(res);
}

```

Poner en `res` el menor de los primeros elementos de cada arreglo (`i` y `j`)

Si se agotó el arreglo `right` y quedaron elementos en `left` sin copiar, copiarlos a `res`


```

/**
 * @post Merges two sorted arrays left and right into one sorted array res
 */
private void merge(Comparable[] left, Comparable[] right, Comparable[] res) {
    // check precondition
    assert isSorted(left);
    assert isSorted(right);
    int i = 0, j = 0, k = 0;
    // Compare and take the smaller element
    while (i < left.length && j < right.length) {
        if (left[i].compareTo(right[j]) <= 0) {
            res[k] = left[i];
            i++;
        } else {
            res[k] = right[j];
            j++;
        }
        k++;
    }
    // Add any leftovers from the left array
    while (i < left.length) {
        res[k] = left[i];
        i++;
        k++;
    }
    // Add any leftovers from the right array
    while (j < right.length) {
        res[k] = right[j];
        j++;
        k++;
    }
    assert isSorted(res);
}

```

Poner en `res` el menor de los primeros elementos de cada arreglo (`i` y `j`)

Si se agotó el arreglo `right` y quedaron elementos en `left` sin copiar, copiarlos a `res`

Si se agotó el arreglo `left` y quedaron elementos en `right` sin copiar, copiarlos a `res`

Mergesort: Análisis de eficiencia

```
/**
 * @post Rearranges the array a in ascending order
 * using the mergesort algorithm.
 */
public void sort(Comparable[] arr) {
    // Base case: arrays of length 0 or 1 are already sorted
    if (arr.length <= 1) {
        return;
    }
    // 1. Divide into two halves
    int mid = arr.length / 2;
    Comparable[] left = copyRange(arr, 0, mid);
    Comparable[] right = copyRange(arr, mid, arr.length);
    // 2. Recursively sort both halves
    sort(left);
    sort(right);
    // 3. Merge back the two sorted halves to arr
    merge(left, right, arr);
    assert isSorted(arr);
}
```

Mergesort: Análisis de eficiencia

```
/**  
 * @post Rearranges the array a in  
 * using the mergesort algorithm  
 */
```

```
public void sort(Comparable[] arr) {  
    // Base case: arrays of length 0 or 1 are already sorted  
    if (arr.length <= 1) {  
        return;  
    }  
    // 1. Divide into two halves  
    int mid = arr.length / 2;  
    Comparable[] left = copyRange(arr, 0, mid);  
    Comparable[] right = copyRange(arr, mid, arr.length);  
    // 2. Recursively sort both halves  
    sort(left);  
    sort(right);  
    // 3. Merge back the two sorted halves to arr  
    merge(left, right, arr);  
    assert isSorted(arr);  
}
```

el tamaño del input n es la cantidad de elementos en `arr`

Mergesort: Análisis de eficiencia

```
/**
 * @post Rearranges the array a in
 * using the mergesort algorithm
 */
public void sort(Comparable[] arr) {
    // Base case: arrays of length 0 or 1 are already sorted
    if (arr.length <= 1) {
        return;
    }
    // 1. Divide into two halves
    int mid = arr.length / 2;
    Comparable[] left = copyRange(arr, 0, mid);
    Comparable[] right = copyRange(arr, mid, arr.length);
    // 2. Recursively sort both halves
    sort(left);
    sort(right);
    // 3. Merge back the two sorted halves to arr
    merge(left, right, arr);
    assert isSorted(arr);
}
```

el tamaño del input n es la cantidad de elementos en `arr`

left tiene aproximadamente $n/2$ elementos

Mergesort: Análisis de eficiencia

```
/**
 * @post Rearranges the array a in
 * using the mergesort algorithm
 */
public void sort(Comparable[] arr) {
    // Base case: arrays of length 0 or 1 are already sorted
    if (arr.length <= 1) {
        return;
    }
    // 1. Divide into two halves
    int mid = arr.length / 2;
    Comparable[] left = copyRange(arr, 0, mid);
    Comparable[] right = copyRange(arr, mid, arr.length);
    // 2. Recursively sort each half
    sort(left);
    sort(right);
    // 3. Merge back the two halves
    merge(left, right, arr);
    assert isSorted(arr);
}
```

el tamaño del input n es la cantidad de elementos en `arr`

left tiene aproximadamente $n/2$ elementos

right tiene aproximadamente $n/2$ elementos

Mergesort: Análisis de eficiencia

```
/**  
 * @post Rearranges the array a in  
 * using the mergesort algorithm  
 */
```

```
public void sort(Comparable[] arr) {  
    // Base case: arrays of length 0 or 1 are already sorted  
    if (arr.length <= 1) {  
        return;  
    }  
    // 1. Divide into two halves  
    int mid = arr.length / 2;  
    Comparable[] left = copyRange(arr, 0, mid);  
    Comparable[] right = copyRange(arr, mid, arr.length);  
    // 2. Recursively sort each half  
    sort(left);  
    sort(right);  
    // 3. Merge back the two halves  
    merge(left, right, arr);  
    assert isSorted(arr);  
}
```

el tamaño del input n es la cantidad de elementos en `arr`

`left` tiene aproximadamente $n/2$ elementos

`right` tiene aproximadamente $n/2$ elementos

`merge` recorre todos los elementos de `left` y `right`, por lo que es $\Theta(n)$

Tiempo de Ejecución

Analicemos el tiempo de ejecución de merge sort en el peor caso:

- La implementación de merge que vimos es $\Theta(n)$
- Su ecuación de recurrencia de merge sort viene dada por:

$$T(0) = c_1$$

$$T(1) = c_2$$

$$T(n) = 2 * T(n/2) + 2 * n$$

llamadas recursivas

merge es n y copiar arreglos es n

Tiempo del MergeSort

Hagamos sustituciones:

$$T(n) = 2 * T(n/2) + 2 * n \quad \text{Paso 1}$$

$$= 4 * T(n/4) + 4 * n \quad \text{Paso 2}$$

$$= 8 * T(n/8) + 6 * n \quad \text{Paso 3}$$

En i sustituciones nos da: $2^i * T(n/2^i) + 2 * i * n$

Obtenemos que: $T(\frac{n}{2^i}) = 1$ cuando: $\frac{n}{2^i} = 1$ Es decir: $i = \log_2 n$

Reemplazando: $2^{\log_2 n} * T(1) + 2 * \log_2 n * n$

$$= n * c_2 + 2 * \log_2 n * n$$

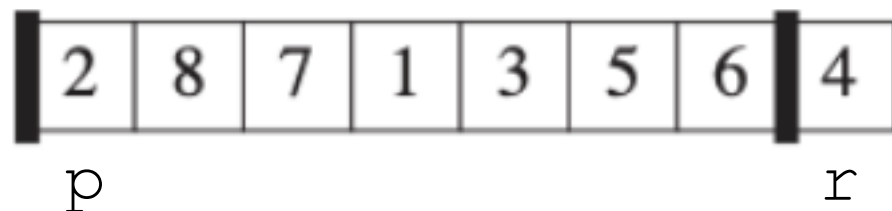
Por lo tanto: $T(n) \in \Theta(n * \log_2 n)$

Mergesort

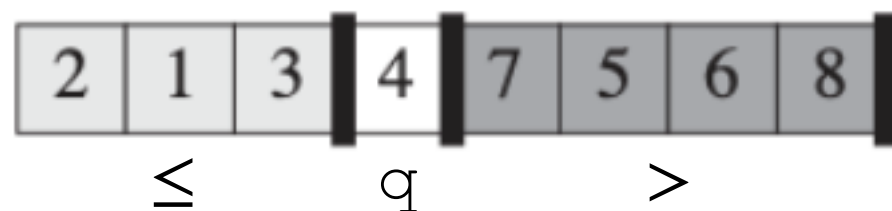
- Es un algoritmo estable (bien implementado)
- Ventaja: En el peor caso es: $\Theta(n * \log_2 n)$
- La implementación que vimos no ordena in-place, es decir, no ordena sobre el mismo arreglo
- Desventaja: Requiere $\Theta(n)$ espacio adicional para crear los arreglos auxiliares

Quicksort: Idea general

- Elegir un elemento pivot en el arreglo
 - En esta versión tomaremos el último elemento como pivot ($A[r]$)



- Particionar el arreglo de modo que el pivot quede en el lugar que le corresponde (q), a la izquierda queden elementos menores o iguales que el pivot, y a la derecha queden elementos mayores que el pivot



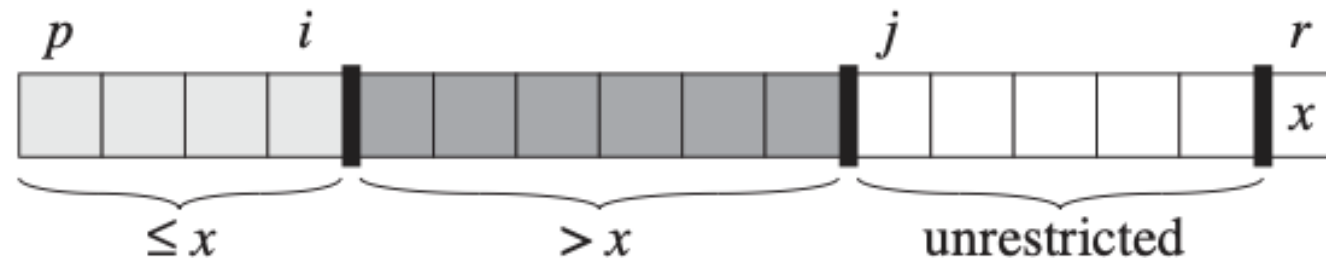
- Ordenar recursivamente las partes izquierda ($A[p \dots q-1]$) y derecha ($A[q+1 \dots r]$)

```
public void sort(Comparable[] a) {  
    sort(a, 0, a.length - 1);  
    assert isSorted(a);  
}
```

```
private void sort(Comparable[] a,  
                  int p, int r) {  
    if (p < r) {  
        int q = partition(a, p, r);  
        // Sort left part a[p .. q-1]  
        sort(a, p, q-1);  
        // Sort right part a[q+1 .. r]  
        sort(a, q+1, r);  
        assert isSorted(a, p, r);  
    }  
}
```

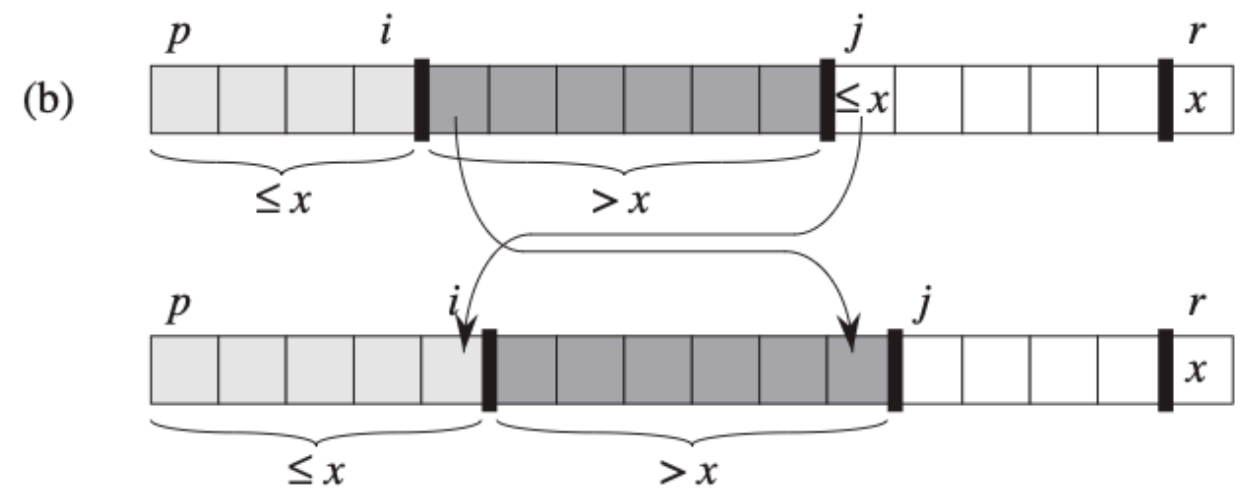
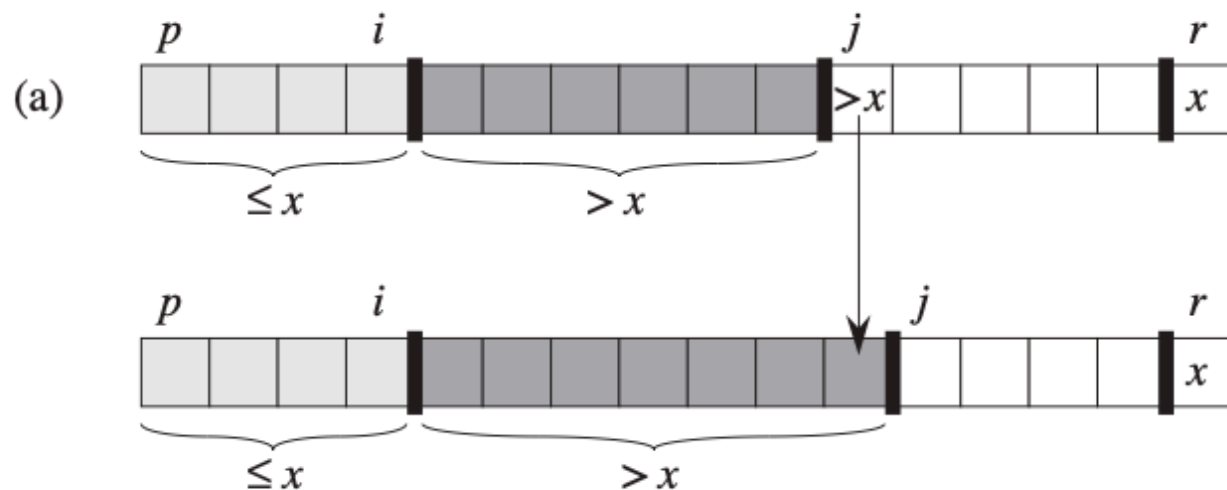
Partition

- Elegir un último elemento como pivot, $x = A[r]$
- Vamos a iterar sobre los elementos del arreglo (con j desde p hasta $r-1$) manteniendo los siguientes invariantes:

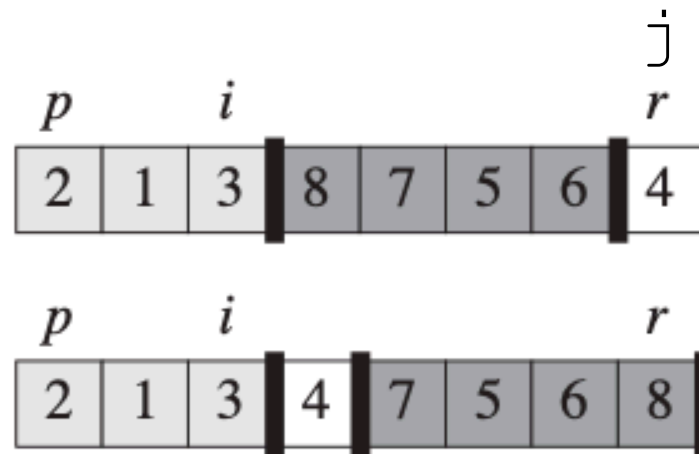


1. If $p \leq k \leq i$, then $A[k] \leq x$.
2. If $i + 1 \leq k \leq j - 1$, then $A[k] > x$.
3. If $k = r$, then $A[k] = x$.

- Tenemos que considerar dos casos para el elemento corriente $A[j]$:
 - (a) Si $A[j] > x$ simplemente pasamos al siguiente j
 - (b) Si $A[j] \leq x$ incrementamos i e intercambiamos $A[j]$ con $A[i]$



Partition



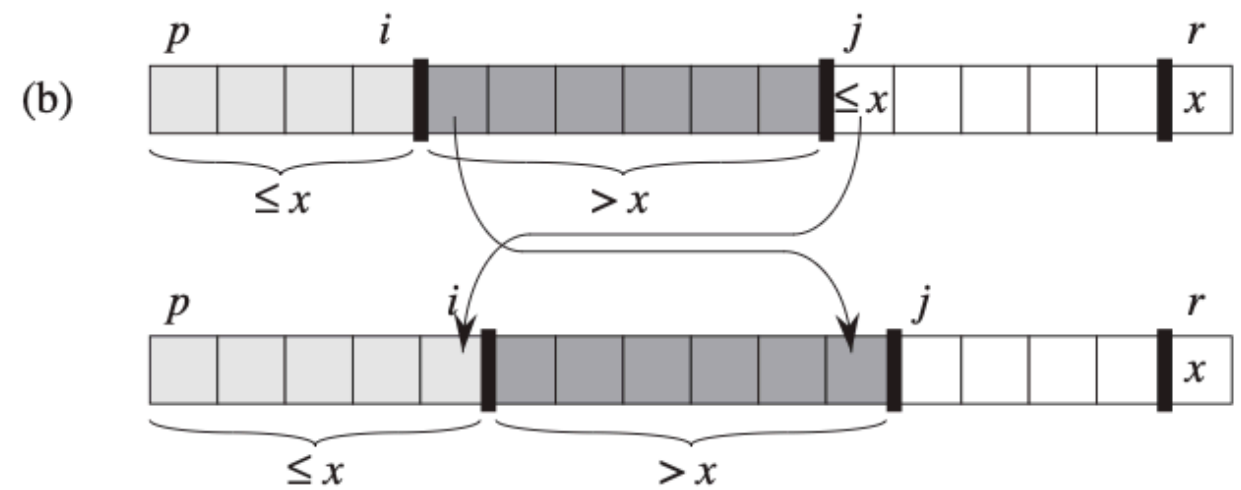
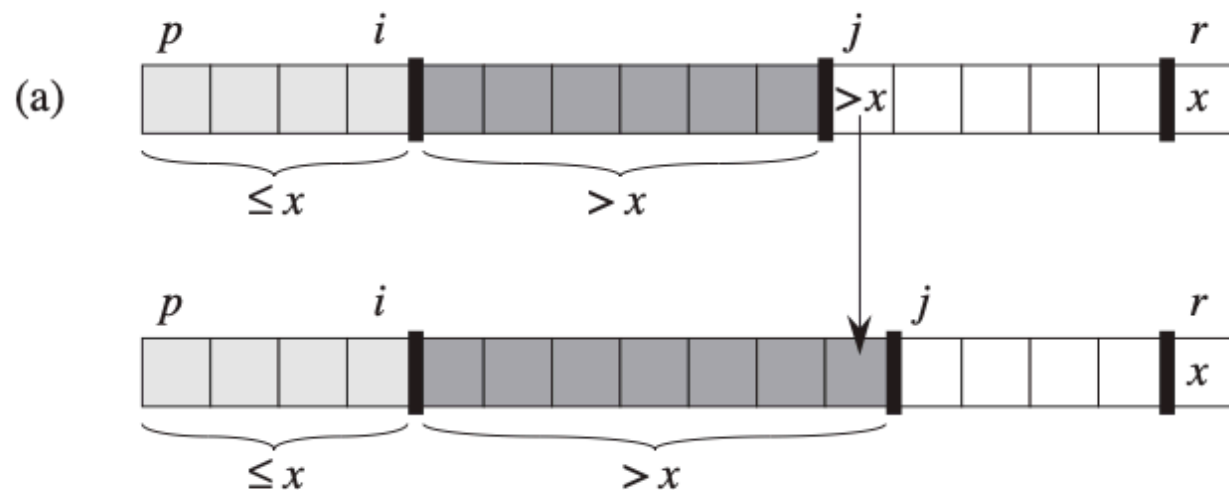
- Por último (cuando $j = r$) sólo nos queda:
 - Intercambiar $A[i+1]$ con $A[r]$ para poner el pivot en su lugar
 - Retornar la posición del pivot $i+1$
- Esta versión de partition es eficiente porque recorre el arreglo una sola vez (es $\Theta(n)$)

Partition: Implementación

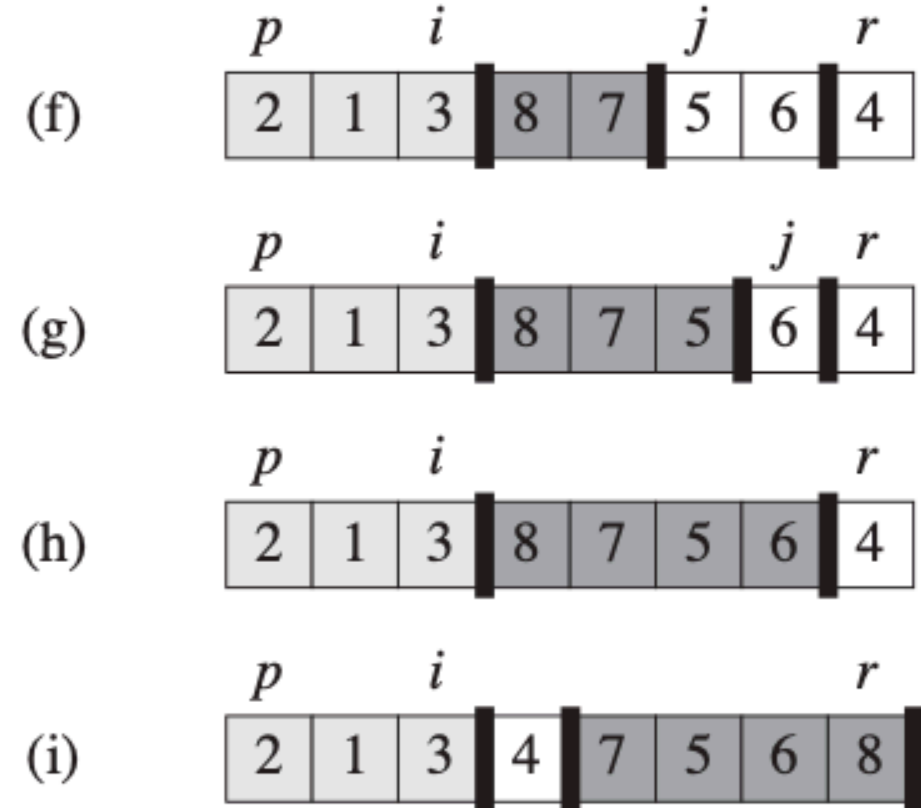
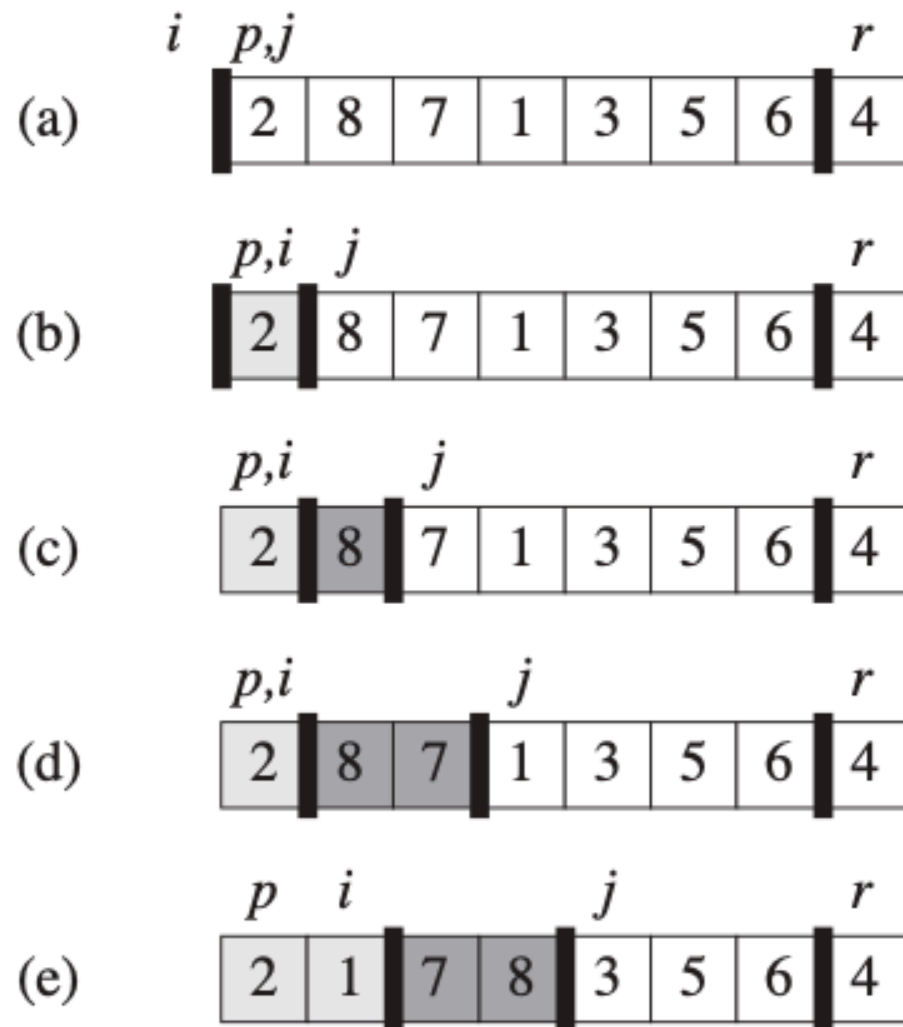
```
/**
 * @post Partition the subarray a[p..r] choosing a[r] as a pivot.
 */
private int partition(Comparable[] a, int p, int r) {
    Comparable x = a[r];
    int i = p - 1;
    for (int j = p; j < r; j++) {
        if (a[j].compareTo(x) <= 0) {
            i++;
            exch(a, i, j);
        }
    }
    // Put x into its position i+1
    exch(a, i+1, r);
    return i+1;
}
```

Partition: Implementación

```
/**
 * @post Partition the subarray a[p..r] choosing a[r] as a pivot.
 */
private int partition(Comparable[] a, int p, int r) {
    Comparable x = a[r];
    int i = p - 1;
    for (int j = p; j < r; j++) {
        if (a[j].compareTo(x) <= 0) {
            i++;
            exch(a, i, j);
        }
    }
    // Put x into its position i+1
    exch(a, i+1, r);
    return i+1;
}
```



Partition: Ejemplo completo



Tiempo de Ejecución

En el peor caso de elección del pivot, la cantidad de elementos se decrementa en uno, es decir:

$$T(0) = c_1$$

$$T(1) = c_2$$

$$T(n) = c + T(n - 1) + n$$

Siempre se elige el elemento más chico o más grande como pivot.

tiempo del partition

Llamada recursiva

Es decir, tenemos: $T(n) \in O(n^2)$

Por ejemplo: [4,1,2,3]

Ej:Correr partition!

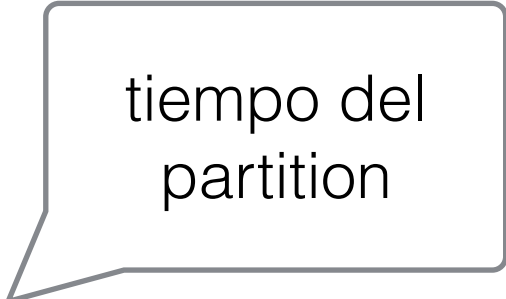
Tiempo de Ejecución

En un buen caso, partition dividirá el arreglo en aproximadamente dos mitades, y obtenemos la misma ecuación de recurrencia que en el mergesort

$$T(0) = c_1$$

$$T(1) = c_2$$

$$T(n) = 2 * T\left(\frac{n}{2}\right) + n$$



tiempo del
partition

Es decir, tenemos: $T(n) \in O(n * \log_2 n)$

En el caso promedio, si usamos un pivot elegido aleatoriamente, partition va a generar algunas divisiones buenas y otras malas

Se puede demostrar que en el caso promedio es $O(n * \log_2 n)$

Quicksort con elección aleatoria del pivot

- Antes de ejecutar el quicksort, vamos a intercambiar el último elemento del arreglo con un elemento aleatorio del arreglo

```
public void sort(Comparable[] a) {  
    // Swap the last element with a random element  
    // to eliminate dependence on input  
    Random random = new Random();  
    int k = random.nextInt(a.length - 1);  
    exch(a, k, a.length - 1);  
    // Quicksort  
    sort(a, 0, a.length - 1);  
    assert isSorted(a);  
}
```

- Intuitivamente, esto aumenta las probabilidades de que el elemento elegido como pivot no sea ni un elemento muy grande ni uno muy chico
- Es decir, las particiones en muchos casos deberían tener una cantidad similar de elementos

Observaciones

- En la practica el Quicksort se comporta mejor que otros algoritmos de sorting
 - En el caso promedio es $O(n * \log_2 n)$
 - El peor caso tiene muy pocas probabilidades de ocurrir
 - Y la constante de proporcionalidad es más pequeña que para Mergesort
- El partition no necesita espacio extra (ordena in-place, es decir, sobre el mismo arreglo)
- No es un algoritmo estable (depende de la implementación del partition)

Cota inferior para Algoritmos de Sorting

Tenemos el siguiente resultado para algoritmos de sorting:

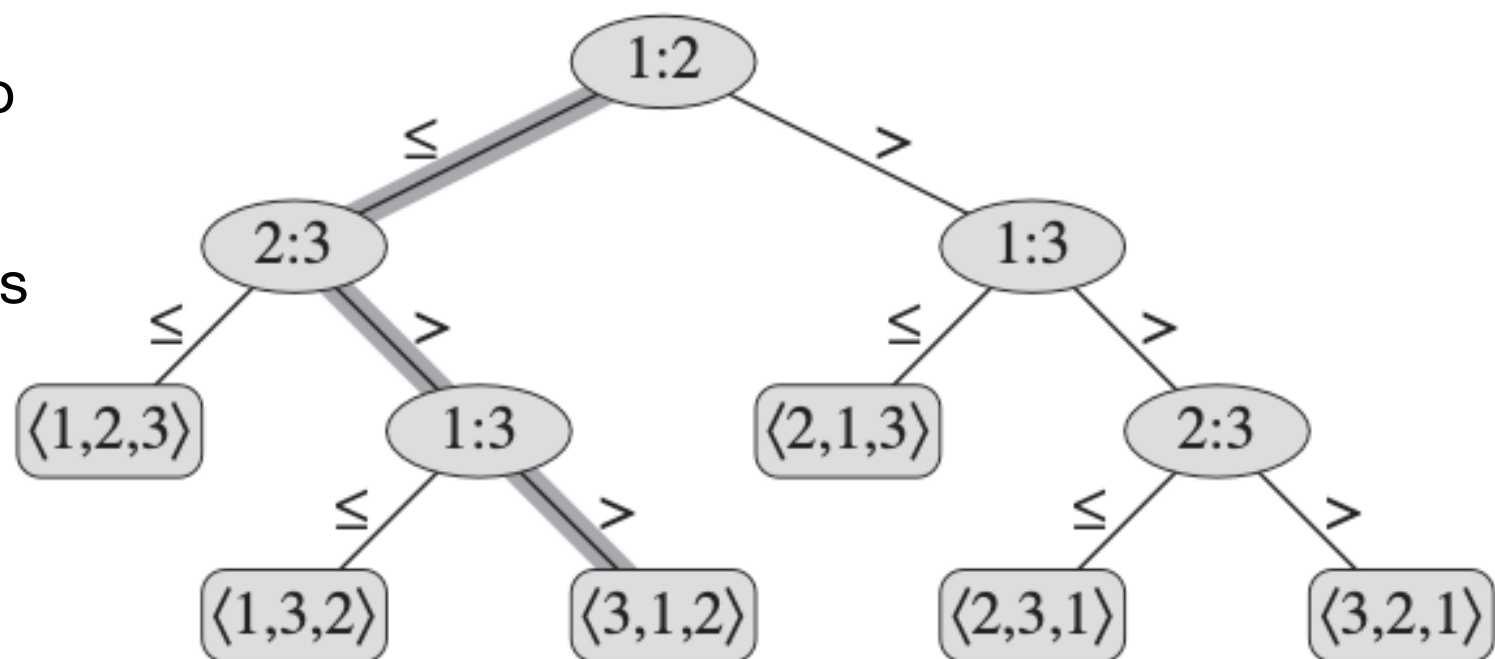
Teorema: Cualquier algoritmo de sorting que utilice comparaciones para ordenar es $\Omega(n * \log n)$

Corolario: Mergesort es un algoritmo basado en comparaciones asintóticamente óptimo (es $\Theta(n * \log_2 n)$)

Cota inferior para sorting:

Prueba

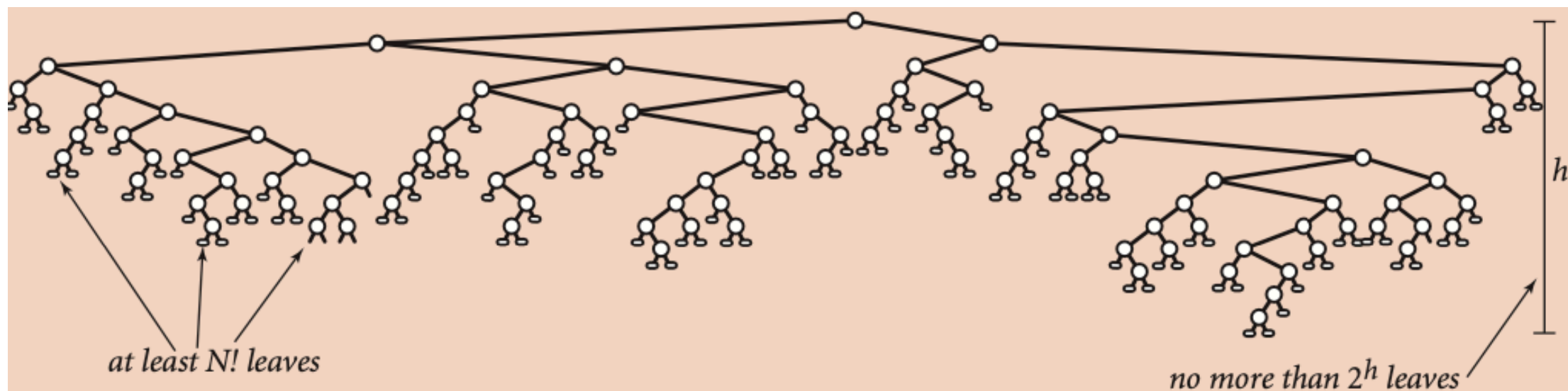
- Podemos modelar el funcionamiento de los algoritmos de sorting que realizan comparaciones con un árbol de decisión
- Nos abstraemos del valor particular de cada elemento y consideramos sólo sus posiciones
- Los nodos internos modelan las comparaciones realizadas por el algoritmo
 - Un nodo anotado con $i:j$ modela que el algoritmo realiza una comparación entre los elementos $a[i]$ y $a[j]$
- Las hojas denotan que el algoritmo llegó a una permutación ordenada del arreglo original
 - Ej: $\langle 3, 1, 2 \rangle$ denota que $a[3] \leq a[1] \leq a[2]$
- Un camino desde la raíz a una hoja representa una ejecución del algoritmo
- Cualquier algoritmo correcto debe producir las $n!$ permutaciones posibles de su entrada
 - O fallaría para las permutaciones que no puede generar



Cota inferior para sorting:

Prueba

- Sea h la altura del árbol con l hojas que modela el comportamiento del algoritmo basado en comparaciones para ordenar n elementos
 - La máxima cantidad de comparaciones que hace el algoritmo es h



- Para que el algoritmo sea correcto debe valer: $n! \leq l$
 - Propiedad: Un árbol de altura h tiene a lo sumo 2^{h-1} hojas: $l \leq 2^h$
 - Luego: $n! \leq 2^h$ y
 - $\log_2(n!) \leq \log_2 2^h$ (tomando logaritmos a ambos miembros)
 - $\log_2(n!) \leq h$ (propiedad: $\log_2(n!) \in \Theta(n * \log_2 n)$)
 - $n * \log_2 n \leq h$
- La cantidad de comparaciones $h \in \Omega(n * \log_2 n)$

Counting sort

- Counting sort asume que los n elementos de entrada están en un rango pequeño $[0..k]$
 - Counting sort no hace comparaciones entre los elementos
 - Si $k \in O(n)$ el algoritmo es $\Theta(n)$
 - Requiere $\Theta(n)$ espacio adicional para el arreglo auxiliar que se usa para contar

- Idea:

- 1. Contar la cantidad de repeticiones de los elementos en un arreglo auxiliar C

	1	2	3	4	5	6	7	8		0	1	2	3	4	5	
A	2	5	3	0	2	3	0	3		C	2	0	2	3	0	1

- 2. Contar la cantidad de elementos menores o iguales en el arreglo C

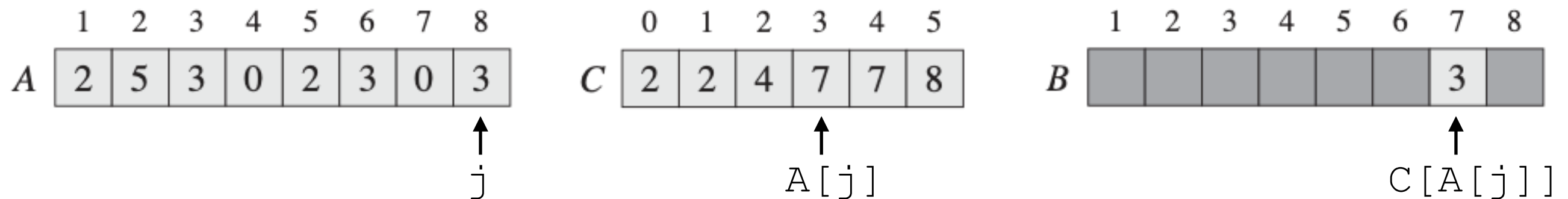
	0	1	2	3	4	5
C	2	2	4	7	7	8

- C nos dice dónde tenemos que ubicar cada elemento de A
- Si hay k elementos menores o iguales a $A[i]$, $A[i]$ va en la posición k del resultado ordenado

Counting sort

- 3. Iterar j desde el final hacia el comienzo del arreglo:
 - El elemento $A[j]$ va en la posición $C[A[j]]$ del resultado
 - Restar 1 a $C[A[j]]$

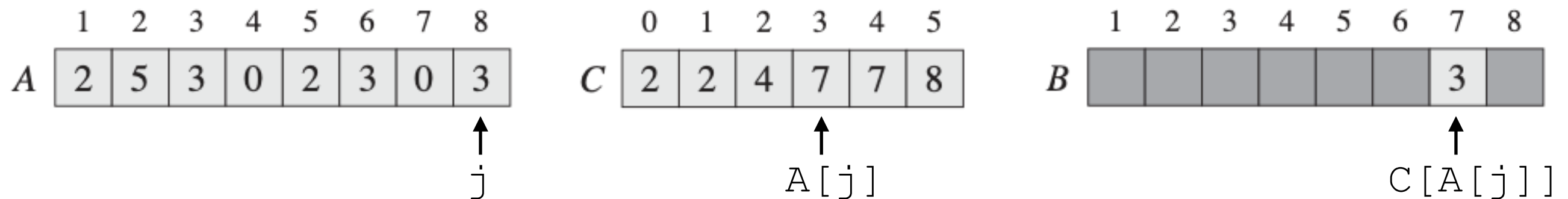
Paso $j=8$:



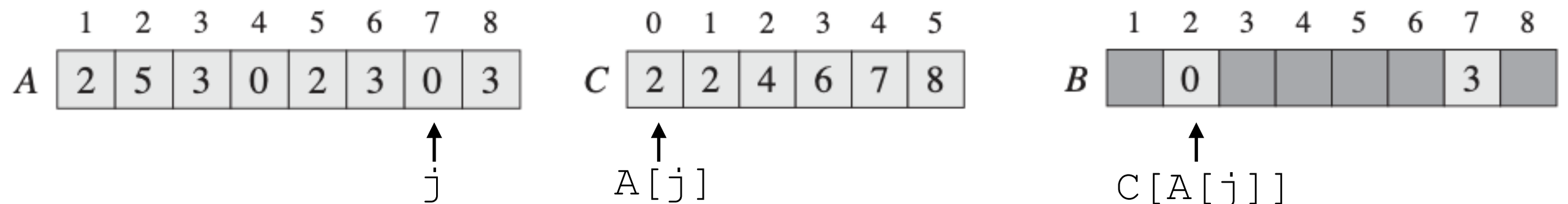
Counting sort

- 3. Iterar j desde el final hacia el comienzo del arreglo:
 - El elemento $A[j]$ va en la posición $C[A[j]]$ del resultado
 - Restar 1 a $C[A[j]]$

Paso $j=8$:



Paso $j=7$:



Y así sucesivamente...

COUNTING-SORT(A, B, k)

```
1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 
```

COUNTING-SORT(A, B, k)

```
1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 
```

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	2	0	2	3	0	1

COUNTING-SORT(A, B, k)

```

1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 

```

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	2	0	2	3	0	1

	0	1	2	3	4	5
C	2	2	4	7	7	8

COUNTING-SORT(A, B, k)

```

1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 

```

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	2	0	2	3	0	1

	0	1	2	3	4	5
C	2	2	4	7	7	8

	1	2	3	4	5	6	7	8
B							3	

	0	1	2	3	4	5
C	2	2	4	6	7	8

COUNTING-SORT(A, B, k)

```

1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 

```

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	2	0	2	3	0	1

	0	1	2	3	4	5
C	2	2	4	7	7	8

	1	2	3	4	5	6	7	8
B							3	

	1	2	3	4	5	6	7	8
B		0					3	

	0	1	2	3	4	5
C	2	2	4	6	7	8

	0	1	2	3	4	5
C	1	2	4	6	7	8

COUNTING-SORT(A, B, k)

```

1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 

```

	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	2	0	2	3	0	1

	0	1	2	3	4	5
C	2	2	4	7	7	8

	1	2	3	4	5	6	7	8
B							3	

	1	2	3	4	5	6	7	8
B		0					3	

	1	2	3	4	5	6	7	8
B		0				3	3	

	0	1	2	3	4	5
C	2	2	4	6	7	8

	0	1	2	3	4	5
C	1	2	4	6	7	8

	0	1	2	3	4	5
C	1	2	4	5	7	8

Tiempo de ejecución

Contar menores
o iguales

Inicialización

$$T(n, k) = n + n + k + n \in \Theta(k + n)$$

Contar
repeticiones

Poner los elementos
en su lugar

COUNTING-SORT(A, B, k)

```
1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 
```

se ejecuta n veces

se ejecuta n veces

se ejecuta k veces

se ejecuta n veces

Tiempo de ejecución

Contar menores
o iguales

Inicialización

$$T(n, k) = n + n + k + n \in \Theta(k + n)$$

Contar
repeticiones

Poner los elementos
en su lugar

COUNTING-SORT(A, B, k)

```
1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 
```

se ejecuta n veces

se ejecuta n veces

se ejecuta k veces

se ejecuta n veces

Si $k \in O(n)$ es $\Theta(n)$

Tiempo de ejecución

Inicialización

Contar menores
o iguales

$$T(n, k) = n + n + k + n \in \Theta(k + n)$$

Contar
repeticiones

Poner los elementos
en su lugar

COUNTING-SORT(A, B, k)

```
1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 
```

se ejecuta n veces

se ejecuta n veces

se ejecuta k veces

se ejecuta n veces

Si $k \in O(n)$ es $\Theta(n)$

Si k es muy grande, el algoritmo es ineficiente!

Tiempo de ejecución

Contar menores
o iguales

Inicialización

$$T(n, k) = n + n + k + n \in \Theta(k + n)$$

Contar
repeticiones

Poner los elementos
en su lugar

COUNTING-SORT(A, B, k)

```
1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 
```

se ejecuta n veces

se ejecuta n veces

se ejecuta k veces

se ejecuta n veces

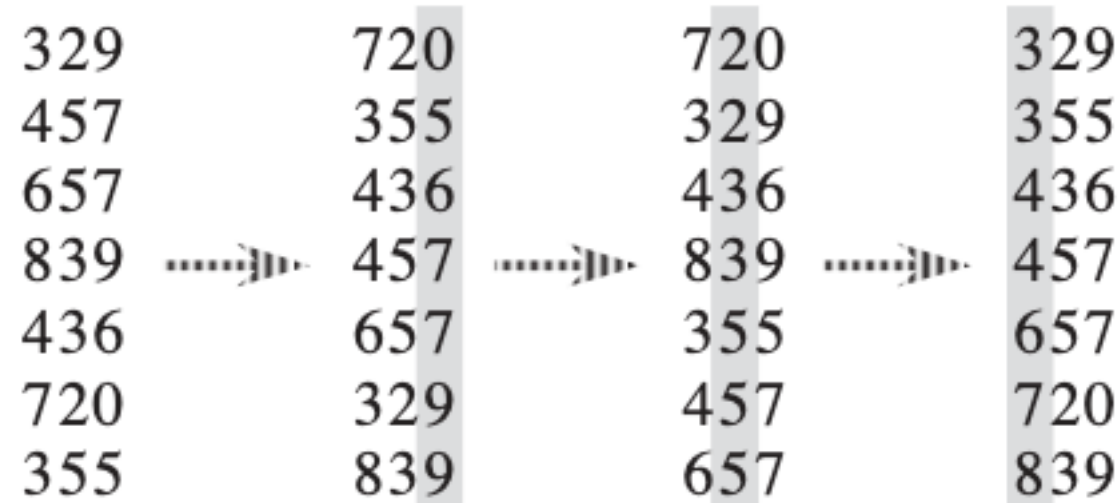
Si $k \in O(n)$ es $\Theta(n)$

Si k es muy grande, el algoritmo es ineficiente!

Esta implementación de counting sort es estable

Radix sort

- Ordenar los números por el primer dígito menos significativo
- Ordenar los números por el segundo dígito menos significativo
- Ordenar los números por el tercer dígito menos significativo
- Y así sucesivamente...



- Para que funcione correctamente requiere usar un algoritmo de sorting estable
- También se puede usar para ordenar cartas, primero ordenando por número y después por palo

Radix sort

RADIX-SORT(A, d)

1 **for** $i = 1$ **to** d

2 use a stable sort to sort array A on digit i

- Podemos usar cualquier algoritmo de ordenamiento estable para implementar Radix sort
 - por ejemplo, Counting sort
- Si el algoritmo usado es $\Theta(n + k)$ (ej. counting sort) radix sort es $\Theta(d * (n + k))$

Actividades

- Leer los capítulos 2, 3, 7, 8 del libro "Introduction to Algorithms, 3rd Edition". T. Cormen, C. Leiserson, R. Rivest, C. Stein. MIT Press. 2009

Bibliografía

- "Algorithms (4th edition)". R. Sedgewick, K. Wayne. Addison-Wesley. 2016
- "Introduction to Algorithms, 3rd Edition". T. Cormen, C. Leiserson, R. Rivest, C. Stein. MIT Press. 2009
- "Data Structures and Algorithms". A. Aho, J. Hopcroft, J. Ullman. Addison-Wesley. 1983